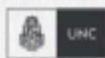


CERTIFICACIÓN UNIVERSITARIA



Manual para el
alumno

Terraform I



**WE WANT
SKILLS!**

mundosE

Manual para el alumno: Terraform I.....	2
Objetivo del encuentro:.....	2
1. ¿Qué es Terraform y por qué laC?.....	3
2. Beneficios de Terraform.....	4
3. Terraform vs. CloudFormation vs. Ansible.....	5
4. Instalación de Terraform.....	6
5. Terraform CLI y Terraform Cloud.....	7
Bibliografía.....	8

Este manual es un complemento de los encuentros sincrónicos de cada módulo. En este encontrarás un resumen de cada tema tratado en el encuentro, así como también, recursos adicionales para poder profundizar sobre los conceptos vistos.

Manual para el alumno: Terraform I

Objetivo del encuentro:

Comprender qué es Terraform, cuáles son sus beneficios frente a otras herramientas de IaC, cómo se instala y cuáles son los entornos y herramientas básicas necesarias para comenzar a trabajar con ella.

Contenido a desarrollar:

- ✓ Concepto de IaC.
- ✓ Declaratividad vs. imperatividad.
- ✓ Beneficios: reproducibilidad, escalabilidad, colaboración, trazabilidad.
- ✓ Terraform vs. AWS CloudFormation (vendor lock-in).
- ✓ Terraform vs. Ansible (configuración vs. provisión de infraestructura).
- ✓ Diferencias en lenguaje y ecosistema.
- ✓ Terraform CLI.
- ✓ Terraform Cloud / Terraform Enterprise (visión conceptual).

1. ¿Qué es Terraform y por qué IaC?

Terraform es una herramienta de infraestructura como código (Infrastructure as Code, IaC) creada por HashiCorp. Se utiliza para automatizar la creación, modificación y destrucción de recursos de infraestructura en diferentes plataformas de nube y entornos on-premises. Su característica principal es que permite **declarar** en archivos de texto cómo queremos que sea la infraestructura, y la herramienta se encarga de que la realidad se ajuste a esa declaración.

La idea central de IaC es transformar algo que históricamente fue manual y propenso a errores (la administración de servidores, redes, bases de datos, etc.) en un proceso reproducible y versionable. Antes de IaC, los administradores realizaban configuraciones a mano en consolas gráficas o mediante comandos. Esto generaba ambientes diferentes, falta de trazabilidad y dificultad para auditar quién cambió qué. Con IaC, en cambio, toda la infraestructura está definida como **código fuente**. Eso significa que se puede guardar en un repositorio Git, revisar mediante *pull requests*, auditar y versionar con el mismo rigor que un programa de software.

Terraform se diferencia de otras herramientas porque adopta un modelo declarativo. En un script imperativo tradicional, el administrador describe **paso a paso** cómo alcanzar un objetivo. Por ejemplo: “crea una máquina virtual, instala un paquete, configura la red”. En Terraform, en cambio, se define el **estado final deseado**: “quiero una máquina virtual con estas características”. A partir de esa definición, Terraform genera un plan de ejecución para converger desde el estado actual al deseado. Este enfoque reduce errores, evita repeticiones innecesarias y permite revertir cambios de manera más sencilla.

Otra característica esencial es que Terraform es **multi-cloud**. Mientras que soluciones como CloudFormation funcionan únicamente con AWS, Terraform es capaz de administrar recursos en AWS, Azure, Google Cloud, Kubernetes, VMware y muchos otros proveedores. Esto se logra gracias a un sistema de **providers**, plugins que permiten a Terraform interactuar con distintas APIs. En la práctica, esto significa que una misma organización puede describir toda su infraestructura de manera unificada, aunque utilice varios proveedores distintos.

Además, Terraform utiliza un **archivo de estado** (*state file*) que refleja cómo está actualmente la infraestructura. Gracias a este mecanismo, puede calcular la diferencia entre lo que está desplegado y lo que se quiere lograr, y aplicar solo los cambios necesarios. Esto permite trabajar de manera incremental y evitar reprovisionar todo cada vez.

El valor de IaC es enorme. Significa que podemos aprender a ver la infraestructura con los mismos principios que aplican al desarrollo de software: repetibilidad, automatización, auditoría, colaboración y trazabilidad. IaC convierte a la

infraestructura en algo predecible y gobernable.

Material complementario:

- HashiCorp. (2025). *What is Infrastructure as Code (IaC)?* <https://www.hashicorp.com/resources/what-is-infrastructure-as-code>
- HashiCorp. (s. f.). *Terraform Basics*. <https://developer.hashicorp.com/terraform/intro>

2. Beneficios de Terraform

Terraform se adoptó masivamente porque resuelve problemas muy comunes en la gestión de infraestructura. Entre sus beneficios más importantes encontramos:

1. **Automatización:** con un archivo `.tf` se pueden desplegar decenas de recursos en cuestión de minutos. Esto elimina procesos manuales lentos y reduce errores humanos.
2. **Reproducibilidad:** las configuraciones son determinísticas. Si se usan los mismos archivos de configuración, el resultado será siempre idéntico, independientemente de dónde se apliquen.
3. **Escalabilidad:** es posible usar la misma configuración para ambientes pequeños o grandes, simplemente ajustando parámetros.
4. **Colaboración:** al ser código, se puede guardar en Git, revisar por pares y aplicar prácticas de control de versiones.
5. **Portabilidad:** gracias a los providers, se puede trabajar con múltiples nubes y servicios desde un mismo conjunto de configuraciones.
6. **Comunidad y módulos:** existen miles de módulos públicos que permiten reutilizar configuraciones validadas por la comunidad o por HashiCorp, acelerando el tiempo de implementación.

Un beneficio adicional es el concepto de **infraestructura inmutable**. En lugar de modificar un recurso en producción, muchas veces se recomienda destruirlo y recrearlo desde cero según la nueva definición. Esto evita que los cambios manuales introduzcan inconsistencias difíciles de rastrear.

Otro punto clave es que Terraform se integra naturalmente en pipelines de CI/CD. Esto significa que la infraestructura puede validarse y desplegarse con el mismo rigor que el software, incluyendo pruebas automáticas, revisiones y despliegues controlados.

Lo importante es entender que estos beneficios no son teóricos: resuelven problemas concretos que todos los equipos de TI enfrentan. Cuando los ambientes no son reproducibles, aparecen errores en producción que no existían en pruebas. Cuando no hay trazabilidad, es imposible saber quién aplicó un cambio. Cuando se depende de un solo proveedor, se pierde flexibilidad. Terraform soluciona todo esto de forma ordenada.

Material complementario:

- Spacelift. (2024). *Terraform Guide*. <https://spacelift.io/blog/terraform>
- HashiCorp. (s. f.). *Terraform Overview*. <https://developer.hashicorp.com/terraform>

3. Terraform vs. CloudFormation vs. Ansible

Comparar herramientas es clave para entender por qué Terraform ganó tanta popularidad.

- **CloudFormation (AWS):** está profundamente integrado con AWS, lo que garantiza soporte inmediato para nuevos servicios. Sin embargo, es exclusivo de AWS. Si la organización quiere usar múltiples proveedores, no es viable. Además, su sintaxis en JSON o YAML puede volverse difícil de mantener en proyectos grandes.
- **Ansible (Red Hat):** es principalmente una herramienta de configuración. Funciona muy bien para instalar software, configurar servidores o desplegar aplicaciones. Es imperativa: describe qué pasos ejecutar. Puede crear infraestructura básica, pero no fue diseñada para eso. Su foco está en el sistema operativo y las aplicaciones.
- **Terraform:** combina declaratividad y portabilidad. Puede crear infraestructura en AWS, Azure, GCP y más. Su sintaxis HCL es más clara y humana que JSON o YAML. Además, ofrece un plan de ejecución (**terraform plan**) que muestra exactamente qué cambios se van a aplicar antes de hacerlos, lo que da seguridad.

En muchos casos, estas herramientas se complementan. Terraform puede crear servidores, redes y bases de datos, mientras que Ansible configura el software dentro de esos servidores. CloudFormation puede ser útil si la organización está 100 % casada con AWS, pero limita la portabilidad.

Para los estudiantes, la lección es clara: no hay una herramienta “mejor” en

términos absolutos, pero Terraform es la opción más versátil y portable.

Material complementario:

- PhoenixNAP. (2024). *Terraform vs. Ansible vs. CloudFormation*.
<https://phoenixnap.com/kb/terraform-vs-ansible>
- HashiCorp. (s. f.). *Terraform Providers*.
<https://developer.hashicorp.com/terraform/providers>

4. Instalación de Terraform

Instalar Terraform es sencillo, pero es importante entender cómo se distribuye y cuáles son las mejores prácticas.

Terraform se entrega como un binario compilado. Para instalarlo manualmente:

1. Descargar el archivo desde la página oficial de HashiCorp.
2. Descomprimirlo.
3. Moverlo a una carpeta incluida en el PATH.
4. Verificar con `terraform version`.

También existen instaladores para distintos sistemas:

- **Linux:** se puede instalar con apt o yum desde repositorios oficiales.
- **macOS:** está disponible mediante Homebrew.
- **Windows:** puede instalarse con chocolatey o scoop.

Otra alternativa es usar **Docker**. Ejecutar Terraform dentro de un contenedor asegura que todos los miembros de un equipo usen exactamente la misma versión, evitando problemas de compatibilidad.

Finalmente, HashiCorp ofrece **Terraform Cloud**, un servicio que elimina la necesidad de instalarlo localmente en algunos casos. Desde allí se pueden ejecutar planes, gestionar el estado y aplicar políticas.

El mensaje para los estudiantes es que la instalación no es una barrera. Lo importante no es el “cómo” técnico de descomprimir un archivo, sino entender que estandarizar versiones es clave para la reproducibilidad.

Material complementario:

- HashiCorp. (s. f.). *Install Terraform*. <https://developer.hashicorp.com/terraform/install>

5. Terraform CLI y Terraform Cloud

Terraform se puede usar de dos formas principales: **local** (con el CLI) o **remota** (con Terraform Cloud).

El **CLI** es la herramienta básica. Se ejecuta desde la consola y permite correr comandos como **terraform init**, **terraform plan** y **terraform apply**. Es ideal para aprender, para proyectos pequeños o para un único administrador. Sin embargo, cuando el equipo crece aparecen problemas:

- El archivo de estado queda en la máquina local.
- Dos usuarios pueden aplicar cambios en paralelo y generar inconsistencias.
- No hay forma sencilla de aplicar políticas de seguridad o revisiones.

Terraform Cloud resuelve estos problemas:

- Centraliza el archivo de estado en un backend seguro.
- Ejecuta los planes de forma remota.
- Se integra con repositorios Git para automatizar despliegues.
- Permite definir políticas que deben cumplirse antes de aplicar cambios.

La analogía con Git es clara: Git se puede usar localmente, pero en equipos se necesita un servicio central como GitHub o GitLab. Lo mismo pasa con Terraform.

Para los estudiantes, entender esta diferencia es fundamental: la elección depende del contexto. Un proyecto pequeño puede manejarse con el CLI local, pero en organizaciones grandes conviene usar Terraform Cloud.

Material complementario:

- HashiCorp. (s. f.). *Terraform Cloud*.
<https://developer.hashicorp.com/terraform/cloud-docs>

Bibliografía

HashiCorp. (s. f.). *Terraform Documentation*. Recuperado de <https://developer.hashicorp.com/terraform>

Spacelift. (2024). *Terraform Guide*. Recuperado de <https://spacelift.io/blog/terraform>

PhoenixNAP. (2024). *Terraform vs. Ansible vs. CloudFormation*. Recuperado de <https://phoenixnap.com/kb/terraform-vs-ansible>